

# ANÁLISIS DEL MÓDULO MANTENIMIENTO DE CONTENEDORES Repositorio:

## /home/runner/work/unirarchivos/unirarchivos Módulo: /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores

### 1. RESUMEN EJECUTIVO

El módulo MantenimientoDeContenedores es una aplicación ASP.NET Core MVC sobre .NET 8 con Entity Framework Core y SQL Server. Su arquitectura se organiza en Models, Controllers, Repositories, Views y un DbContext central. Implementa un flujo maestro-transaccional orientado al taller de mantenimiento: primero configura catálogos base (clientes, contenedores, especialidades, técnicos y tareas) y después ejecuta transacciones de ingreso y previaje por especialidad.

#### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/MantenimientoDeContenedores.csproj:1-13
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Program.cs:6-24
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Views/Home/Index.cshtml:12-72

### 2. ARQUITECTURA TÉCNICA

#### 2.1 Stack

- ASP.NET Core MVC.
- EF Core SqlServer.
- SQL Server como persistencia.
- Bootstrap + CSS propio para UI.

#### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/MantenimientoDeContenedores.csproj:1-13
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Views/Shared/\_Layout.cshtml:7-9, 59-62
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/wwwroot/css/site.css:26-360

#### 2.2 Configuración y arranque

Program.cs registra el DbContext y los repositorios SQL mediante inyección de dependencias. Además, al iniciar, asegura la existencia de la base de datos y crea el esquema con SQL imperativo si aún no existe.

#### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Program.cs:10-24
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Program.cs:46-189

### 3. MODELOS DE DATOS

#### 3.1 CtClienteMtto (maestro de clientes)

##### Propiedades:

- CodCliente: clave primaria, código del cliente.
- NombreCliente: nombre del cliente.
- CodDpto: centro de costo/departamento.
- ImpMovMo: importe de mano de obra sin garantía.
- ImpMovMo2: importe de mano de obra con garantía.
- Activo: estado lógico.

##### Rol funcional:

Es el maestro base para relacionar contenedores y también contiene tarifas que apuntan a futuras reglas de costeo.

#### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Models/CtClienteMtto.cs:5-31
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Data/MantenimientoDbContext.cs:24-55

### 4. DB CONTEXT, ESQUEMA Y RELACIONES

MantenimientoDbContext expone DbSet para clientes, contenedores, especialidades, técnicos, relación técnico-especialidad, ingresos, previajes y tareas.

#### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Data/MantenimientoDbContext.cs:13-20

## 5. CONTROLADORES Y FUNCIONALIDAD

Los controladores principales son ClientesMttoController, ContenedoresController, EspecialidadesMttoController, EspecialidadesEmpleadoController, TareasMttoController, IngresosContenedorController y PreviajesContenedorController. Todos siguen el patrón Index/Create/Edit, y el de previajes concentra la lógica de negocio más rica.

### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Controllers/ClientesMttoController.cs:16-84
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Controllers/PreviasContenedorController.cs

## 6. REPOSITORIOS Y ACCESO A DATOS

Cada agregado tiene interfaz, implementación SQL y versión InMemory. En producción se registran las implementaciones SQL por DI. Se usan AsNoTracking, Include/ThenInclude y SaveChanges como patrón base.

### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Program.cs:12-19
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Repositories/SqlPreviasContenedorRepository.cs

## 7. ESTRUCTURA DE VISTAS E INTERFAZ

La aplicación tiene layout compartido, home con árbol de navegación de maestros y transacciones, tablas index, formularios create/edit y una UI dinámica importante en el previaje.

### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Views/Shared/\_Layout.cshtml:12-63
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Views/Home/Index.cshtml:12-88
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Views/PreviasContenedor/Create.cshtml:137-147

## 8. TRANSACCIONES PRINCIPALES

Las transacciones implementadas son ingreso de contenedor y previaje. No se detectó flujo de salida del contenedor.

### Referencias:

- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Controllers/IngresosContenedorController.cs
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Controllers/PreviasContenedorController.cs
- /home/runner/work/unirarchivos/unirarchivos/MantenimientoDeContenedores/Views/Home/Index.cshtml:53-72

## 9. CONCLUSIÓN

El módulo está orientado a maestros + ingreso + previaje, con reglas de integridad sólidas y evidencias de alcance futuro no completado, sobre todo en detalle de tareas, costeo y salida final del contenedor.